University of Perpignan
C. Legrand

L3 Computer Science
Year 2026

Lab Work 7 – Security

Code Injection

Ethical and Legal Disclaimer

The techniques presented in this lab work are studied within a **strictly academic context** in order to understand the mechanisms used by attackers and thus be better able to defend against them.
Any use of these techniques outside the educational context is illegal and punishable by criminal penalties (Chapter III of the French Penal Code, Articles 323-1 to 323-8 – *Offences against Automated Data Processing Systems*):

- Unauthorized access to or remaining in an ADPS (Art. 323-1): **3 years** imprisonment and **€100,000** fine
- Obstructing the operation of an ADPS (Art. 323-2): **5 years** imprisonment and **€150,000** fine
- Fraudulent introduction of data into an ADPS (Art. 323-3): **5 years** imprisonment and **€150,000** fine

Penalties increased to **7 years** and **€300,000** when the targeted system processes personal data operated by the State, and up to **10 years** and **€300,000** for organized crime (Art. 323-4-1). *Penalties updated by the LOPMI law no. 2023-22 of January 24, 2023.*

Setup with Docker

For this lab work, the environment (Apache + PHP + MySQL server) is provided as Docker containers. No virtual machine is needed.

Prerequisite: install Docker

On Ubuntu/Debian:

```
# Install Docker
sudo apt update
sudo apt install -y docker.io docker-compose-v2
# Add your user to the docker group
sudo usermod -aG docker $USER
# Log out and log back in for the group
# change to take effect
```

On macOS/Windows: install Docker Desktop.

Starting the environment

```
# Navigate to the lab directory
cd travaux-diriges/04-injection-code/td07/

# Start the containers (first time: ~2 min)
docker compose up -d

# Check that everything is running
docker compose ps
```

On first launch, Docker will:

- build an Apache+PHP container with the `mysqli` extension,
- start a MySQL 8.0 container with the `l2info` database,
- automatically load the tables and data from `bd.sql`.

Accessing the web pages

The lab files are accessible in your browser at:

`http://localhost:8080/td7/`

For example: `http://localhost:8080/td7/page-5-commentaires.php`

Stopping and cleaning up

```
# Stop the containers (data is preserved)
docker compose stop

# Restart
docker compose start

# Stop and remove everything (containers + data)
docker compose down -v
```

Warning

In the PHP files, the MySQL server is accessible via the hostname `"db"` (the Docker service name), not `"localhost"`. This is normal Docker behavior: each service runs in a separate container, identified by its name.

Exercise 1 (HTML Form Manipulation)**.** Consider a web shopping application whose HTML form is as follows (file `achat-iphone5.html`):

```html
<html>
<body>

<form method="post" action="enregistre-achat.php">
Produit: iphone5 <br/>
Prix : 500 euros <br/>
Quantit&eacute <input type="text" name ="quantite">
  (Quantite max est 90) <br/>
<input type="hidden" name="prix" value="500" >
<input type="submit" value="buy">
</form>

</body>
</html>
```

The PHP script `enregistre-achat.php` records the purchase in the database using the values sent by the form.

1. Analyze the HTML form. Identify the vulnerability related to how the price is sent to the server.
2. Propose a method to buy an iPhone for €5 instead of €500.
3. Propose a server-side fix to prevent this attack.

Key Takeaway

Hidden fields and any client-side data can be modified by the user. Validation must always be performed server-side.

Course Reminder

There are three types of XSS:

- **Reflected XSS**: the payload is included in the request (URL, GET/POST parameter) and immediately reflected back in the response without being stored.
- **Stored XSS**: the payload is saved on the server (database, file) and displayed to all subsequent visitors of the page.
- **DOM-based XSS**: the payload is interpreted directly by client-side JavaScript, without passing through the server.

Exercise 2 (Cross-Site Scripting (XSS))**.**

a) Stored XSS – JavaScript alert

Consider the page `page-5-commentaires.php` whose code is as follows:

```html
<html>
<H1> Computer Security  </H1>
<IMG src="img_security.jpeg"> Computer security...
<BR/><hr/>
<h3> Visitors comments :</h3>
```

```php
<?php
$conn = new mysqli("db", "l2info", "l2info", "l2info");
if ($conn->connect_error) {
  die("Connection failed: " . $conn->connect_error);
}

if( isset($_POST['login']) && isset($_POST['commentaire']) )
{
    $sql = 'INSERT INTO Commentaires(login,commentaire)
      VALUES("' . $_POST['login']. '","'
      . $_POST['commentaire'].'");';
    $result0 = $conn->query($sql);
}

$sql = 'SELECT id,login, commentaire FROM Commentaires
  ORDER BY id DESC LIMIT 4 ;';
$result = $conn->query($sql);

if ($result->num_rows > 0) {
    echo "<ul>";
    while($row = $result->fetch_assoc()) {
        echo "<li>";
        echo "[" . $row["login"]. "]: "
          . $row["commentaire"]."<br>";
        echo "</li>";
    }
    echo "</ul>";
} else {
    echo "No comments.";
}
$conn->close();
?>
<BR/><hr/><BR/>
<form name="postcomment" method="POST"
  action="page-5-commentaires.php">
Pseudo: <input name="login"> <BR/>
Comment: <input type="text" name="commentaire"><BR/>
<input type="submit">
</form>
</html>
```

1. Explain what this page does.
2. Propose an injection in the comment field so that a JavaScript alert is displayed when the page loads.

b) Stored XSS – defacement

Using the same page, inject fraudulent HTML to make a fake "References" section appear with a link to a fictitious website.

c) Reflected XSS via GET

Consider the following PHP code:

```
<html>
<body>
<?php
if( isset($_GET['UserInput'])){
  $out ='you entered: "'
    . $_GET['UserInput'] . '".';
}
else {
  $out = '<form method="GET">
    enter something here:';
  $out .= '<input name="UserInput" size="50">';
  $out .= '<input type="submit">';
  $out .= '</form>';
}
print $out;
?>
</body>
</html>
```

Submit an input that triggers a JavaScript alert in the result.

d) Reflected XSS via POST

Now consider the following code:

```
<html>
<body>
<?php
if( isset($_POST['UserInput'])){
  $out ='you entered: "'
    . $_POST['UserInput'] . '".';
}
else {
  $out='No data was sent to the script';
}
print $out;
?>
</body>
</html>
```

Build an external HTML page containing a hidden POST form that automatically submits an XSS payload to this page.

e) DOM-based XSS

Consider the following HTML page:

```
<html>
<body>
<div id="welcome"></div>
<script>
  var name = document.location.hash.substring(1);
  document.getElementById("welcome").innerHTML
    = "Welcome " + name;
</script>
</body>
</html>
```

1. Craft a URL that exploits this vulnerability to execute JavaScript.
2. Explain why this type of XSS is harder to detect server-side.

Warning

Stored XSS is the most dangerous because it affects all visitors to the page, without requiring a crafted link.

Course Reminder

Difference between XSS and SQL injection:

- **XSS** injects malicious code into the *victim's browser* (client-side). The attack targets the website's users.
- **SQL injection** injects malicious code into the *server's database* (server-side). The attack directly targets the site's data.

Exercise 3 (SQL Injection).

a) Analyzing the vulnerable code

Consider the page `page-transaction-sql.php`:

```
<html>
<?php
$servername = "db";
$username = "l2info";
$password = "l2info";
$dbname = "l2info";

$conn = new mysqli($servername, $username,
  $password, $dbname);
if ($conn->connect_error) {
  die("Connection failed: "
    . $conn->connect_error);
}
echo 'Name: "' . $_POST['UserInput'].'"<BR/>';
$sql = 'SELECT montant FROM Transaction WHERE nom="'
  . $_POST['UserInput']  . '";';
echo "SQL Query=" . $sql . "<BR/>";
$result = $conn->query($sql);
echo "Result:<BR/>";
if ($result->num_rows > 0) {
  while($row = $result->fetch_assoc()) {
    echo "Name: ". $_POST['UserInput']
      ." Amount:" . $row["montant"]."<br/>";
  }
} else {
  echo "0 results";
}
$conn->close();
?>
</html>
```

And the form `injection-sql.html`:

```html
<html>
<body>
<form name="exploit-sql" method="POST"
  action="http://localhost:8080/td7/page-transaction-sql.php">
<input name="UserInput">
<input type="submit">
</form>
</body>
</html>
```

1. Explain how the SQL query is built. Identify the vulnerability.

b) Display the entire table

Propose an injection to display the entire `Transaction` table. Explain the resulting query step by step.

c) Attempting a DROP TABLE

Attempt to delete the `Test` table via SQL injection. Explain why this may fail with MySQL.

d) UNION-based

Extract the data from the `Test` table (column `val`) using a `UNION SELECT` injection.

Hint: start by determining the number of columns in the original `SELECT` by using `ORDER BY 1`, `ORDER BY 2`, etc. until you get an error. Then construct the `UNION SELECT` with the correct number of columns.

e) Blind SQL injection

Now suppose the page no longer displays the query results but only shows "Transaction found" or "No transaction".

1. How could you determine whether the MySQL version is 5.x? (concept of *boolean-based blind SQL injection*)
2. What would be the principle for extracting data character by character?

Key Takeaway

SQL injection works because the query mixes SQL code and user data in the same string. The solution is to separate them using **prepared statements**.

Exercise 4 (Countermeasures)**. a) Fixing XSS**

Modify the code of `page-5-commentaires.php` to prevent XSS injection. Which PHP function escapes special HTML characters? Provide the corrected code for the comment display line.

b) Fixing SQL injection

Rewrite `page-transaction-sql.php` using PDO prepared statements instead of `mysqli` concatenation. Provide the complete corrected code.

c) Additional defenses

1. What is the `Content-Security-Policy` HTTP header and how does it protect against XSS?
2. What is an `HttpOnly` cookie and why is it useful?
3. Why is the principle of least privilege important for the database?

Key Takeaway

Defense in depth combines multiple mechanisms: output escaping, prepared statements, CSP, `HttpOnly` cookies, input validation.